

Developers can use System.Text.Json to serialize type hierarchies securely

Developers can use System.Text.Json to serialize type hierarchies securely - eiriktsarpalis, GitHub.

- Published: 2022-01-13
- Original: <https://github.com/dotnet/runtime/issues/63747>
- Preserved from: <https://github.com/dotnet/runtime/issues/63747> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Developers can use System.Text.Json to serialize type hierarchies securely

- Repository: dotnet/runtime
- Opened by: eiriktsarpalis
- Opened: 2022-01-13
- State: closed

Body

Background and Motivation

Serializing type hierarchies (aka polymorphic serialization) has been a feature long requested by the community (cf. #29937, #30083). We took a stab at [producing a polymorphism feature](#) during the .NET 6 development cycle, ultimately though the feature was cut since we eventually reached the conclusion that unconstrained polymorphism does not meet our security bar, under any circumstance.

For context, polymorphic serialization has long been associated with security vulnerabilities. More specifically, unconstrained polymorphic serialization can result in accidental data disclosure and unconstrained polymorphic deserialization can result in

remote code execution when used with untrusted input. See the [BinaryFormatter security guide](#) for an explanation of such vulnerabilities.

We acknowledge that serializing type hierarchies is an important feature, and we are committed to delivering a *secure* implementation in a future release of System.Text.Json. This implies that we will be releasing a brand of polymorphism that is restricted by design, requiring users to explicitly opt-in to supported subtypes.

Basic Polymorphism

At the core of the design is the introduction of the `JsonDerivedType` attribute:

```
[JsonDerivedType(typeof(Derived))]  
public class Base  
{  
    public int X { get; set; }  
}  
  
public class Derived : Base  
{  
    public int Y { get; set; }  
}
```

This configuration enables polymorphic serialization for `Base`, specifically when the runtime type is `Derived`:

```
Base value = new Derived();  
JsonSerializer.Serialize<Base>(value); // { "X" : 0, "Y" : 0 }
```

Note that this does not enable polymorphic *deserialization* since the payload would roundtripped as `Base`:

```
Base value = JsonSerializer.Deserialize<Base>(@"{ \"X\" : 0, \"Y\" : 0 }");  
value is Derived; // false
```

Polymorphism using Type Discriminators

To enable polymorphic *deserialization*, users need to specify a *type discriminator* for the derived class:

```

[JsonDerivedType(typeof(Base), typeDiscriminatorId: "base")]
[JsonDerivedType(typeof(Derived), typeDiscriminatorId: "derived")]
public class Base
{
    public int X { get; set; }
}

public class Derived : Base
{
    public int Y { get; set; }
}

```

Which will now emit JSON along with type discriminator metadata:

```

Base value = new Derived();
JsonSerializer.Serialize<Base>(value); // { "$type" : "derived", "X" : 0, "Y" : 0 }

```

which can be used to deserialize the value polymorphically:

```

Base value = JsonSerializer.Deserialize<Base>(@"{ \"$type\" : \"derived\", \"X\" : 0,
\"Y\" : 0 }");
value is Derived; // true

```

Type discriminator identifiers can also be integers, so the following form is valid:

```

[JsonDerivedType(typeof(Derived1), 0)]
[JsonDerivedType(typeof(Derived2), 1)]
[JsonDerivedType(typeof(Derived3), 2)]
public class Base { }

JsonSerializer.Serialize<Base>(new Derived2()); // { "$type" : 1, ... }

```

Mixing and matching configuration

It is possible to mix and match type discriminator configuration for a given type hierarchy

```
[JsonPolymorphic]
[JsonDerivedType(typeof(Derived1))]
[JsonDerivedType(typeof(Derived2), "derived2")]
[JsonDerivedType(typeof(Derived3), 3)]
public class Base
{
    public int X { get; set; }
}
```

resulting in the following serializations:

```
var json1 = JsonSerializer.Serialize<Base>(new Derived1()); // { "X" : 0, "Y" : 0 }
var json2 = JsonSerializer.Serialize<Base>(new Derived2()); // { "$type" : "derived2",
"X" : 0, "Z" : 0 }
```

Customizing Type Discriminators

It is possible to customize the property name of the type discriminator metadata like so:

```
[JsonPolymorphic(CustomTypeDiscriminatorPropertyName = "$case")]
[JsonDerivedType(typeof(Derived1), "derived1")]
public class Base
{
    public int X { get; set; }
}
```

resulting in the following JSON:

```
JsonSerializer.Serialize<Base>(new Derived1()); // { "$case" : "derived1", "X" : 0, "Y" :
0 }
```

Unknown Derived Type Handling

Consider the following type hierarchy:

```
[JsonDerivedType(typeof(DerivedType1))]
public class Base { }
public class DerivedType1 : Base { }
public class DerivedType2 : Base { }
```

Since the configuration does not explicitly opt-in support for `DerivedType2`, attempting to serialize instances of `DerivedType2` as `Base` will result in a runtime exception:

```
JsonSerializer.Serialize<Base>(new DerivedType2()); // throws NotSupportedException
```

The default behavior can be tweaked using the `JsonUnknownDerivedTypeHandling` enum, which can be specified like so:

```
[JsonPolymorphic(UnknownDerivedTypeHandling =  
    JsonUnknownDerivedTypeHandling.FallBackToBaseType)]  
[JsonDerivedType(typeof(DerivedType1))]  
public class Base { }  
  
JsonSerializer.Serialize<Base>(new DerivedType2()); // serialize using the contract for  
`Base`
```

The `FallBackToNearestAncestor` setting can be used to fall back to the contract of the nearest declared derived type:

```
[JsonPolymorphic(UnknownDerivedTypeHandling =  
    JsonUnknownDerivedTypeHandling.FallBackToNearestAncestor)]  
[JsonDerivedType(typeof(MyDerivedClass))]  
public interface IMyInterface { }  
public class MyDerivedClass : IMyInterface { }  
  
public class TestClass : MyDerivedClass { }  
  
JsonSerializer.Serialize<IMyInterface>(new TestClass()); // serializes using the contract  
for `MyDerivedClass`
```

It should be noted that falling back to the nearest ancestor admits the possibility of diamond ambiguity:

```
[JsonPolymorphic(UnknownDerivedTypeHandling =  
JsonUnknownDerivedTypeHandling.FallBackToNearestAncestor)]  
[JsonDerivedType(typeof(MyDerivedClass))]  
public interface IMyInterface { }  
  
public interface IMyDerivedInterface : IMyInterface { }  
public class MyDerivedClass : IMyInterface { }  
  
public class TestClass : MyDerivedClass, IMyDerivedInterface { }  
  
JsonSerializer.Serialize<IMyInterface>(new TestClass()); // throws NotSupportedException
```

Configuring Polymorphism via the Contract model

For use cases where attribute annotations are impractical or impossible (large domain models, cross-assembly hierarchies, hierarchies in third-party dependencies, etc.), it should still be possible to configure polymorphism using the [JSON contract model](#):

```

public class MyPolymorphicTypeResolver : DefaultJsonTypeInfoResolver
{
    public override JsonTypeInfo GetTypeInfo(Type type, JsonSerializerOptions options)
    {
        JsonTypeInfo jsonTypeInfo = base.GetTypeInfo(type, options);
        if (jsonTypeInfo.Type == typeof(Base))
        {
            jsonTypeInfo.PolymorphismOptions =
                new JsonPolymorphismOptions
                {
                    TypeDiscriminatorPropertyName = "_case",
                    UnknownDerivedTypeHandling =
                        JsonUnknownDerivedTypeHandling.FallBackToNearestAncestor,
                    DerivedTypes =
                        {
                            new JsonDerivedType(typeof(DerivedType1)),
                            new JsonDerivedType(typeof(DerivedType2), "derivedType2"),
                            new JsonDerivedType(typeof(DerivedType3), 42),
                        }
                }
        }

        return jsonTypeInfo;
    }
}

```

Additional details

- Polymorphic serialization only supports derived types that have been explicitly opted in via the `JsonDerivedType` attribute. Undeclared runtime types will result in a runtime exception. The behavior can be changed by configuring the `JsonPolymorphicAttribute.UnknownDerivedTypeHandling` property.
- Polymorphic configuration specified in derived types is not inherited by polymorphic configuration in base types. These need to be configured independently.
- Polymorphic hierarchies are supported for both classes and interface types.
- Polymorphism using type discriminators is only supported for type hierarchies that use the default converters for objects, collections and dictionary types.
- Polymorphism is supported in metadata-based sourcegen, but not fast-path sourcegen.

API Proposal

```

namespace System.Text.Json.Serialization
{
    [AttributeUsage(AttributeTargets.Class | AttributeTargets.Interface, AllowMultiple =
false, Inherited = false)]
    public sealed class JsonPolymorphicAttribute : JsonAttribute
    {
        public string? TypeDiscriminatorPropertyName { get; set; }
        public bool IgnoreUnrecognizedTypeDiscriminators { get; set; }
        public JsonUnknownDerivedTypeHandling UnknownDerivedTypeHandling { get; set; }
    }

    [AttributeUsage(AttributeTargets.Class | AttributeTargets.Interface, AllowMultiple =
true, Inherited = false)]
    public class JsonDerivedTypeAttribute : JsonAttribute
    {
        public JsonDerivedTypeAttribute(Type derivedType);
        public JsonDerivedTypeAttribute(Type derivedType, string typeDiscriminatorId);
        public JsonDerivedTypeAttribute(Type derivedType, int typeDiscriminatorId);

        public Type DerivedType { get; }
        public object? TypeDiscriminatorId { get; }
    }

    public enum JsonUnknownDerivedTypeHandling
    {
        FailSerialization = 0, // Default, fail serialization on undeclared derived type
        FallBackToBaseType = 1, // Fall back to the base type contract
        FallBackToNearestAncestor = 2 // Fall back to the nearest declared derived type
contract (admits diamond ambiguities in cases of interface hierarchies)
    }
}

```

```

namespace System.Text.Json.Metadata;

public partial class JsonTypeInfo
{
    public JsonPolymorphismOptions? PolymorphismOptions { get; set; } = null;
}

public class JsonPolymorphismOptions
{
    public JsonPolymorphismOptions();

    public ICollection<JsonDerivedType> DerivedTypes { get; }

    public bool IgnoreUnrecognizedTypeDiscriminators { get; set; } = false;
    public JsonUnknownDerivedTypeHandling UnknownDerivedTypeHandling { get; set; } =
        JsonUnknownDerivedTypeHandling.Default;
    public string TypeDiscriminatorPropertyName { get; set; } = "$type";
}

// Use a dedicated struct instead of ValueTuple that handles type checking of the
// discriminator id
public struct JsonDerivedType
{
    public JsonDerivedType(Type derivedType);
    public JsonDerivedType(Type derivedType, int typeDiscriminatorId);
    public JsonDerivedType(Type derivedType, string typeDiscriminatorId);

    public Type DerivedType { get; }
    public object? TypeDiscriminatorId { get; }
}

```

Anti-Goals

- No support for serializing open hierarchies (as specified in #29937).
- No support for deserializing open hierarchies.

Progress

- [x] Bring [prototype branch](#) up to date.
- [x] Implementation & testing.
- [x] API proposal & review.
- [x] Conceptual documentation & blog posts.

Comments

ghost, 2022-01-13

Tagging subscribers to this area: @dotnet/area-system-text-json See info in [area-owners.md](#) if you want to be subscribed. <details> <summary>Issue Details</summary> <hr />

Background and Motivation

Serializing type hierarchies (aka polymorphic serialization) has been a long requested feature by the community (cf. #29937, #30083). We took a stab at [producing a polymorphism feature](#) during the .NET 6 development cycle, ultimately though the feature was cut since we eventually reached the conclusion that unconstrained polymorphism does not meet our security bar, under any circumstance.

For context, polymorphic serialization has long been associated with security vulnerabilities. More specifically, unconstrained polymorphic serialization can result in accidental data disclosure and unconstrained polymorphic deserialization can result in remote code execution when used with untrusted input. See the [BinaryFormatter security guide](#) for an explanation of such vulnerabilities.

We acknowledge that serializing type hierarchies is an important feature, and we are committed at delivering a *secure* implementation in a future release of System.Text.Json. This implies that we will be releasing a brand of polymorphism that is restricted by design, requiring users to explicitly opt-in to supported subtypes.

Examples

At the core of the design is the introduction of `JsonKnownType` attribute that can be applied to type hierarchies like so:

```

[JsonTypeDiscriminator("$type")]
[JsonKnownType(typeof(Derived1), "derived1")]
[JsonKnownType(typeof(Derived2), "derived2")]
public class Base
{
    public int X { get; set; }
}

public class Derived1 : Base
{
    public int Y { get; set; }
}

public class Derived2 : Base
{
    public int Z { get; set; }
}

```

This allows roundtrippable polymorphic serialization using the following schema:

```

var json1 = JsonSerializer.Serialize<Base>(new Derived1()); // { "$type" : "derived1",
"X" : 0, "Y" : 0 }
var json2 = JsonSerializer.Serialize<Base>(new Derived2()); // { "$type" : "derived2",
"X" : 0, "Z" : 0 }

JsonSerializer.Deserialize<Base>(json1) is Derived1; // true
JsonSerializer.Deserialize<Base>(json2) is Derived2; // true

```

Type discriminators can optionally be omitted from the contract like so:

```

[JsonKnownType(typeof(Derived1))]
[JsonKnownType(typeof(Derived2))]
public class Base
{
    public int X { get; set; }
}

```

resulting in the following serializations:

```

var json1 = JsonSerializer.Serialize<Base>(new Derived1()); // { "X" : 0, "Y" : 0 }
var json2 = JsonSerializer.Serialize<Base>(new Derived2()); // { "X" : 0, "Z" : 0 }

```

Note however that the particular contract does not support deserialization.

See also <https://github.com/dotnet/runtime/issues/30083#issuecomment-861524767> for details on the most recent API proposal.

Goals

- Users should be able to serialize and deserialize type hierarchies using type discriminators.
- Users must explicitly specify all supported subtypes for a given base type, following the approach used by `DataContractSerializer`'s [KnownTypeAttribute](#).
- Users can optionally omit type discriminators from the serialized value, losing the ability to roundtrip the serialized value.
- Users should be able to specify type hierarchies for types via APIs in `JsonSerializerOptions`.
- Users should be able to specify type hierarchies [via customized JSON contracts](#).
- Type discriminators can either be of type string or type int, but a single hierarchy cannot mix and match types.
- We should be able to reuse the polymorphism infrastructure when delivering related features, such as <https://github.com/dotnet/runtime/issues/55744>.

Anti-Goals

- No support for serializing open hierarchies (as specified in #29937).
- No support for deserializing open hierarchies.

Open Questions

- Should we support polymorphic serialization for collection types?
- How should we handle serialization of types in the hierarchy not explicitly opted in (adopt contract of nearest ancestor vs. throwing an exception). The former approach suffers from diamond ambiguity in the context of interface hierarchies.
- How should we handle serialization of values of the base type if the base type has not been explicitly opted in?

Progress

- [] Bring [prototype branch](#) up to date.
- [] API proposal & review.
- [] Implementation & testing.
- [] Conceptual documentation & blog posts.

Author:	eiriktsarpalis
Assignees:	eiriktsarpalis
Labels:	

```
area-System.Text.Json , User Story , Priority:2 , Cost:L , Team:Libraries
```

```
</td> </tr> <tr> <th align="left">Milestone:</th> <td>7.0.0</td> </tr> </table>
</details>
```

SCLDGit, 2022-01-13

I'm curious for more explanation by the open question "Should we support polymorphic serialization for collection types?".

If it simply means "Should we allow users to serialize a collection of objects of various types that all derive from IType?" (e.g. List<IShape>) , then I can't possibly imagine a scenario where the answer would be no.

eiriktsarpalis, 2022-01-13

If it simply means "Should we allow users to serialize a collection of objects of various types that all derive from IType?" (e.g. List) , then I can't possibly imagine a scenario where the answer would be no.

That's right. The issue primarily is that collections serialize as JSON arrays. As such introducing a type discriminator would serve to complicate the contract, i.e. it would likely require nesting the array payload inside an object with metadata:

```
{ "$type" : "derivedList", "$values" : [1,2,3,4,5] }
```

There is precedent for such encodings in the reference preservation feature.

IFYates, 2022-01-13

unconstrained polymorphic serialization can result in accidental data disclosure (Anti-Goal) No support for serializing open hierarchies

I completely understand these arguments and decisions, but I think it's a shame that this option is taken away from the developer. An opt-in to serializing open hierarchies would have quickly bridged a large gap from Newtonsoft.Json (from what I've seen in other comments) and could prove useful in some diagnostic situations. I've taken someone's example for a write-only polymorphic converter, which fills my needs but feels overdue for official support.

For the above examples, would it be too much to also support an inverse of the attribute:

```
[JsonTypeDiscriminator("$type")]
public class Base
{
    public int X { get; set; }
}

[JsonKnownDerivedType("derived2")]
public class Derived1 : Base
{
    public int Y { get; set; }
}
```

This provides a clean solution for when you don't own the `Base` class. I feel like it might complicate deserialization, but even for a serialize-only approach, it feels better than having to maintain a central list of type mappings. (Assuming I've understood "Users should be able to specify type hierarchies for types via APIs in JsonSerializerOptions" correctly)

SCLDgit, 2022-01-13

It does feel a bit insulting to have a much-requested feature completely disallowed simply because people *might* misuse it. Even if we had to toss it in an unsafe block, it would be nice to have simple does-what-it-says-on-the-box JSON serialization/deserialization for the (very common) case where security is not a concern. I've used JSON as a quick, simple data interchange format (most recently to store hierarchical scene data for rendering software) more times than I can count, and not having to rely on Newtonsoft.JSON would be great for those cases.

eiriktsarpalis, 2022-01-13

For the above examples, would it be too much to also support an inverse of the attribute:

Such an attribute would likely need to explicitly specify the base type, since otherwise that would be ambiguous: was it meant for `Base` ? or `object` ? or some interface that `Derived1` happens to implement?

In any case, it should be possible to work around not owning the base type by specifying the hierarchy via `JsonSerializerOptions`.

jeffhandley, 2022-01-13

We acknowledge and understand the frustration that folks have with open hierarchies being disallowed. Our intent with publishing these goals and anti-goals is to help bring clarity to expectations and engage in these conversations. This should help us find a viable path forward together.

It does feel a bit insulting to have a much-requested feature completely disallowed simply because people *might* misuse it.

The challenge with open hierarchies isn't that people *might* misuse it, it's that *any* use of it is insecure. There are scenarios in which that vulnerability is masked by defenses *elsewhere* in a system, including the "I wrote this code and it's only ever going to run on my machine with my own data" scenario, but we cannot design this feature set with reliance on outside defenses. We must approach this design knowing that foundational security is paramount and then enable as many scenarios as we can with features that are still usable. We want to be clear about how we're approaching the problem space.

By designing with security first, we can keep building new capabilities outward to incrementally shrink the set of scenarios that cannot be accomplished. But if we start with features that are vulnerable by design, then we'll inevitably reach a point where it's difficult to use the features *securely*. That was the fate of `BinaryFormatter`, which is planned to be [_removed_ from .NET](#) as a result.

We want to hear more about scenarios where open hierarchies have been used, and explore approaches for addressing those scenarios without relying on open hierarchies as an implementation detail. Thank you for helping us keep this moving forward!

eiriktsarpalis, 2022-01-14

And, for what it's worth, it should still be possible to implement unconstrained polymorphism using custom converters. I've seen a few examples like that on the internet (including a couple of threads in this repo), although I wouldn't want to share links to them here.

IFYates, 2022-01-14

Such an attribute would likely need to explicitly specify the base type, since otherwise that would be ambiguous: was it meant for `Base`? or `object`? or some interface that `Derived1` happens to implement?

Yes, I originally wrote `[JsonKnownDerivedType(typeof(Base), "derived1")]`, but went for the stripped-down example. I agree that forcing to define the base class is clearer. Not sure the implications of specifying a base higher up the hierarchy.

In any case, it should be possible to work around not owning the base type by specifying the hierarchy via `JsonSerializerOptions`.

Can you show an example? This sounds like (unless I've misunderstood) requiring the types to all be registered when the serializer is configured (e.g., in an OWIN startup). While that feels correct if I'm tying together both things outside of my control, when I own the derived type, it feels smelly compared to your example of owning the the base class.

Secondarily related, would this structure work to expose the necessary in the derived type?

```

[JsonTypeDiscriminator("$type")]
[JsonKnownType(typeof(IExposeY), "derived1")] // Serialize based on interface;
acknowledging that deserialization is impossible
public class Base
{
    public int X { get; set; }
}

public class Derived1 : Base, IExposeY
{
    public int Y { get; set; } // From IExposeY
}

```

Symbai, 2022-01-14

And, for what it's worth, it should still be possible to implement unconstrained polymorphism using custom converters.

But we don't want hand crafted solutions. If we would, the request wouldn't have so many up votes where people are telling you (and everyone posting his custom solution) this over and over again. The problem is rather simple: If the JSON .NET team make any change in the future because they are not aware of everyone's custom solution, and this change breaks this custom solution. But this custom solution has been used for so long that so many JSON (files for example) already exist. Then we have the worst case scenario ever. As a developer we want reliability, we want something where we can be sure it also works in 5 years or more. And we want it simply, something to opt-in. Something that works everywhere the same. No matter which developer wrote the code for it.

Newtonsoft JSON provides it. System.Text.JSON does not. And since .NET 3.1 we are asking for it and now that it has been delayed for so many times telling us to use custom converters is a bit odd. If that was ever an option, we would have used it already. But we are not, we are waiting for an official solution. Please keep this in mind, thank you.

eiriktsarpalis, 2022-01-14

But we are not, we are waiting for an official solution.

Just to be absolutely clear, there will not be an officially supported way to do unconstrained polymorphic serialization in the foreseeable future. We are always open to offering better extensibility points that make such features easier to build as extensions, but the onus for doing this will always be on the developer/ecosystem.

SCLDGit, 2022-01-14

This is an especially annoying situation for those of us on the desktop app side of the equation who use JSON as a data storage or round-trip save format. The only security

concern to us, at least as far as I understand it, is that someone could alter an on-disk JSON file that our software has generated that could do something malicious when loaded back into the system. If this is indeed the case (and please correct me if it's not), then there are already myriad data integrity validation tools at our disposal, including cryptographic hashing and signing, that make this concern a non issue. In every scenario where I want to serialize and deserialize JSON, I am in control of the data from end to end. Ignoring this use case in favor of only addressing the concerns of anonymous web transfers is ignoring a large chunk of the equation. No one is suggesting that security not be the prime concern by default, but if every library design approached its domain problem with the idea that developers should be absolutely disallowed from getting ourselves into potential trouble, even if it's opt-in, nothing would ever get done in the field of software development. Please reconsider this stance. Many of us don't require this level of hand holding.

jeremyVignelles, 2022-01-14

I agree with the fact that unconstrained polymorphism is a security issue, and as far as I understand, the only real counter-arguments here are that it worked before and that it's easy to dump process memory and load it back.

If it can be done with the proper extensions method, and that it's as useful as you say, I'm sure there will be an open-source package that will fill the gap, without tempting new devs that don't understand the security implications of this. What do you think?

ishepherd, 2022-01-14

I'm sure there will be an open-source package that will fill the gap,

This seems logical. As S.T.J necessarily evolves slowly, an open source extensions package it is a natural idea. S.T.J should provide good enough extension points to let this package be built.

Because the package is not by MS, MS cannot rightly be blamed for the vulns it introduces.

quixoticaxis, 2022-01-14

Am I wrong in my assumption that the issue being spoken about arises not from unconstrained polymorphism per se, but from the fact that traditionally multiple popular libraries provide the serialization facilities in ways that do not enforce class invariants and de-facto do binary deserialization, and also from the common issue (as I subjectively see it) of providing loose invariants throughout the ecosystem? If everything is deserialized via constructor calls and most of the types in use guarantee invariants, the open polymorphism is not an issue, right?

On topic tl;dr: as an optional mode, open hierarchies may be allowed if the library is set to use constructors.

eiriktsarpalis, 2022-01-14

provide the serialization facilities in ways that do not enforce class invariants

Can you clarify a bit more what you mean by class invariants?

Symbai, 2022-01-14

If it can be done with the proper extensions method, and that it's as useful as you say, I'm sure there will be an open-source package that will fill the gap

There are already open source packages which provide polymorphism support. But I found they don't work in all cases or aren't updated anymore. This is what can(!) happen to all third party packages. If you rely on them and got thousands of JSON files and it doesn't work anymore and you have to use another package perhaps and all of your previous JSON becomes invalid.. good luck. Not to say that any further updates to System.Text.Json can break it as well. I don't think this an appropriated solution for a serious developer/company but that's just my opinion.

quixoticaxis, 2022-01-14

> provide the serialization facilities in ways that do not enforce class invariants Can you clarify a bit more what you mean by class invariants?

@eiriktsarpalis I mean the set of assertions that stay true through the lifetime of the object, often enforced by the constructors.

I can understand that the class that provides public setters with no validation for properties of non specific types (for example, the class uses signed integers to model natural numbers) can be insecurely deserialized even without polymorphism, but I fail to imagine an example of a class that 1) would be insecure to recreate on deserialization through constructor, and 2) would not have other obvious design issues. A quick Internet search didn't help.

eiriktsarpalis, 2022-01-17

I would say that's an orthogonal concern. Polymorphism is achievable while still honoring class invariants in the type hierarchy. TL;DR the security concern around polymorphic deserialization is allowing the payload to specify the arbitrary .NET type it wants to deserialize into, which can be exploited for remote code execution. A few real examples of such exploits can be found [here](#).

quixoticaxis, 2022-01-17

I would say that's an orthogonal concern. Polymorphism is achievable while still honoring class invariants in the type hierarchy. TL;DR the security concern around polymorphic deserialization is allowing the payload to specify the arbitrary .NET type it wants to deserialize into, which can be exploited for remote code execution. A few real examples of such exploits can be found [here](#).

@eiriktsarpalis , thank you for the link. Oh, I see, we're speaking about the scenario when the code author is clearly stating the intent to handle literally anything by deserializing `object` , but does not think about the consequences. Is it really an issue? One of the issues with binary deserialization is the ability to re-create a thoughtfully designed object of some type (that does not omit validation itself) in a state that is not reachable throughout in-memory object's lifetime. Json deserialization is completely devoid of this flaw, or at least puts the burden of security (via validation of inputs) on the class' author. Maybe I'm missing something, but I'm not certain I agree with lots of people (who are trying to `Deserialize<MyBase>(payload)`) paying the price of additional code bloat to secure that one guy who decided to "deserialize *any* object, `cause what can go wrong?"

10 further comment(s) not preserved.